

AIAC – ASSIGNMENT – 2.3

Name:E.Anjani Batch – 32 Hno: 2303A52419

Task1:

E generate Python code that reads a text file and counts word frequency, then explains the code.

#CODE

```
import os

# Create a dummy text file for demonstration
file_path = 'sample_text.txt'
with open(file_path, 'w') as f:
    f.write("This is a sample text file. This file contains sample words.")

print(f"Created '{file_path}' for demonstration.")

... Created 'sample_text.txt' for demonstration.

import collections
import re

def count_word_frequency(file_path):
    """
    Reads a text file and counts the frequency of each word.

    Args:
        file_path (str): The path to the text file.

    Returns:
        collections.Counter: A Counter object where keys are words and values are their frequencies.
    """
    word_list = []
    with open(file_path, 'r') as f:
        text = f.read()
        # Convert to lowercase and remove punctuation
        text = text.lower()
        # Use regex to find all words (alphanumeric sequences)
        words = re.findall(r'\b\w+\b', text)
        word_list.extend(words)

    return collections.Counter(word_list)

# Use the function with the dummy file
word_counts = count_word_frequency(file_path)

print("Word Frequencies:")
for word, count in word_counts.most_common(10): # Display top 10 most common words
    print(f"{word}: {count}")
```

#OUTPUT

```
... Word Frequencies:
this: 2
sample: 2
file: 2
is: 1
a: 1
text: 1
contains: 1
words: 1
```

Code Explanation

1. Creating a Dummy File (First Cell):

- `import os`: Imports the `os` module, though not strictly needed for this specific file creation, it's good practice for file operations.
- `file_path = 'sample_text.txt'`: Defines the name of the file.
- `with open(file_path, 'w') as f:`: Opens a file named `sample_text.txt` in write mode (`'w'`). The `with` statement ensures the file is automatically closed.
- `f.write(...)`: Writes the specified string content into the file.

2. Counting Word Frequencies (Second Cell):

- `import collections`: Imports the `collections` module, specifically for `collections.Counter` which is excellent for counting hashable objects.
- `import re`: Imports the `re` module for regular expressions, used to find words.
- `def count_word_frequency(file_path):`: Defines a function that takes the file path as an argument.
- `with open(file_path, 'r') as f:`: Opens the specified file in read mode (`'r'`).
- `text = f.read()`: Reads the entire content of the file into the `text` variable.
- `text = text.lower()`: Converts all characters in the text to lowercase to ensure that words like "The" and "the" are counted as the same word.
- `words = re.findall(r'\b\w+\b', text)`: Uses a regular expression to find all words.
 - `\b`: Word boundary (ensures whole words are matched).
 - `\w+`: Matches one or more alphanumeric characters (letters, numbers, and underscore). This effectively extracts words and ignores punctuation.
- `word_list.extend(words)`: Adds all found words to a list.
- `return collections.Counter(word_list)`: Creates a `Counter` object from the list of words. The `Counter` automatically counts the occurrences of each unique word.
- `word_counts = count_word_frequency(file_path)`: Calls the function with our dummy file and stores the result.
- `for word, count in word_counts.most_common(10):`: Iterates through the top 10 most common words and their counts using the `most_common()` method of the `Counter` object, then prints them.

Observation :

During this task, Gemini successfully generated Python code that could read a text file, process its contents, and count word frequencies using a dictionary. The tool also provided a clear explanation of how file I/O, string splitting, and dictionary operations work. Gemini accurately elaborated each part of the code, such as file handling, loop logic, and counting mechanism. The generated code ran efficiently in Google Colab without any modifications. This demonstrated Gemini's capability to produce both functional programs and human-readable explanations suitable for beginners.

TASK 2:



generate a program that PERFORMS FILE OPERATIONS that Creates a text file, Writes sample text, Reads and displays the content

#CODE AND OUTPUT

```
# Step 1: Create a text file and write sample text to it
file_name = 'my_new_sample_file.txt'
sample_text = "Hello from the new file!\nThis is a second line of text for demonstration."

with open(file_name, 'w') as file:
    file.write(sample_text)

print(f"Successfully created '{file_name}' and wrote content to it.")
```

Successfully created 'my_new_sample_file.txt' and wrote content to it.



```
# Step 2: Read the content of the file and display it

# Ensure the file_name variable is accessible, or redefine it if this cell is run independently
file_name = 'my_new_sample_file.txt'

print(f"Reading content from '{file_name}':\n")
with open(file_name, 'r') as file:
    content = file.read()
    print(content)
```

```
...
Reading content from 'my_new_sample_file.txt':
Hello from the new file!
This is a second line of text for demonstration.
```

EXPLANATION:

 Gemini

Code Explanation

1. Creating and Writing to the File (First Code Cell):

- `file_name = 'my_new_sample_file.txt'`: Defines the name of the text file to be created.
- `sample_text = "..."`: Stores the text that will be written into the file.
- `with open(file_name, 'w') as file:`: This line opens the specified file in write mode ('w'). If the file doesn't exist, it will be created. If it does exist, its content will be truncated (emptied) before writing. The `with` statement ensures the file is properly closed even if errors occur.
- `file.write(sample_text)`: Writes the `sample_text` string into the opened file.
- `print(...)`: Confirms that the file has been created and written to.

2. Reading and Displaying the File Content (Second Code Cell):

- `file_name = 'my_new_sample_file.txt'`: Redefines `file_name` to ensure it's available if this cell is executed out of order, though in a sequential run, it would use the variable from the previous cell.
- `with open(file_name, 'r') as file:`: Opens the specified file in read mode ('r').
- `content = file.read()`: Reads the entire content of the file and stores it in the `content` variable as a single string.
- `print(content)`: Displays the read content to the console.

#OBSERVATION:

Cursor AI was able to generate a clean and functional script for creating, writing, and reading text files. The tool also provided inline documentation and improved file-handling structure when refactoring was requested. Compared to traditional editors, Cursor AI's interface helped in quickly previewing changes and understanding the workflow. The generated code executed correctly and produced the expected text output. Overall, Cursor AI performed well for practical file operations and showed strong refactoring ability on request.

TASK 3:



GENERATE A PYTHON CODE THAT READ A CSV FILE AND CALCULATE MEAN, MIN AND , MAX

#CODE

 # Step 1: Create a dummy CSV file for demonstration

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [24, 27, 22, 32, 29],
    'Score': [85.5, 91.2, 78.9, 94.0, 88.3],
    'Height_cm': [165, 180, 170, 175, 160]
}

dummy_df = pd.DataFrame(data)

csv_file_name = 'sample_data.csv'
dummy_df.to_csv(csv_file_name, index=False)

print(f"Successfully created '{csv_file_name}' with sample data.")
```

... Successfully created 'sample_data.csv' with sample data.

```

[6]
/ 0s
# Step 2: Read the CSV file and calculate mean, min, and max
import pandas as pd

csv_file_name = 'sample_data.csv' # Ensure the file name is defined

try:
    df = pd.read_csv(csv_file_name)
    print(f"\nSuccessfully read '{csv_file_name}'. First 5 rows:\n")
    print(df.head())

    print("\nCalculating statistics for numerical columns:")

    # Select only numerical columns for calculations
    numerical_cols = df.select_dtypes(include=['number'])

    if not numerical_cols.empty:
        # Calculate Mean
        mean_values = numerical_cols.mean()
        print("\nMean values:\n", mean_values)

        # Calculate Minimum
        min_values = numerical_cols.min()
        print("\nMinimum values:\n", min_values)

        # Calculate Maximum
        max_values = numerical_cols.max()
        print("\nMaximum values:\n", max_values)
    else:
        print("No numerical columns found to calculate statistics.")

except FileNotFoundError:
    print(f"Error: The file '{csv_file_name}' was not found.")
except Exception as e:
    print(f"An error occurred: {e}")

```

#OUTPUT:

```

Successfully read 'sample_data.csv'. First 5 rows:

```

	Name	Age	Score	Height_cm
0	Alice	24	85.5	165
1	Bob	27	91.2	180
2	Charlie	22	78.9	170
3	David	32	94.0	175
4	Eve	29	88.3	160

```

Calculating statistics for numerical columns:

Mean values:
  Age      26.80
  Score    87.58
  Height_cm 170.00
  dtype: float64

Minimum values:
  Age      22.0
  Score    78.9
  Height_cm 160.0
  dtype: float64

Maximum values:
  Age      32.0
  Score    94.0
  Height_cm 180.0
  dtype: float64

```

Code Explanation

1. Creating a Dummy CSV File (First Code Cell):

- `import pandas as pd`: Imports the pandas library, which is essential for working with tabular data like CSV files.
- `data = {...}`: Creates a Python dictionary to hold sample data, including both string and numerical values.
- `dummy_df = pd.DataFrame(data)`: Converts the dictionary into a pandas DataFrame.
- `csv_file_name = 'sample_data.csv'`: Defines the name of the CSV file to be created.
- `dummy_df.to_csv(csv_file_name, index=False)`: Writes the DataFrame to a CSV file. `index=False` prevents pandas from writing the DataFrame index as a column in the CSV.

2. Reading CSV and Calculating Statistics (Second Code Cell):

- `import pandas as pd`: Imports pandas again (good practice if cells are run independently).
- `csv_file_name = 'sample_data.csv'`: Specifies the name of the CSV file to read.
- `try...except`: A block to handle potential `FileNotFoundError` if the CSV doesn't exist or other general errors.
- `df = pd.read_csv(csv_file_name)`: Reads the CSV file into a pandas DataFrame named `df`.
- `print(df.head())`: Displays the first 5 rows of the DataFrame to show its content.
- `numerical_cols = df.select_dtypes(include=['number'])`: This crucial step selects only columns that have numerical data types (integers and floats) from the DataFrame. This ensures that statistics like mean, min, and max are only calculated for appropriate columns.
- `if not numerical_cols.empty`: Checks if there are any numerical columns to process.
- `mean_values = numerical_cols.mean()`: Calculates the arithmetic mean for each numerical column.
- `min_values = numerical_cols.min()`: Finds the minimum value in each numerical column.
- `max_values = numerical_cols.max()`: Finds the maximum value in each numerical column.
- The `print` statements then display these calculated statistics.

#OBSERVATION:

Gemini effectively handled CSV processing in Google Colab by suggesting the use of Python's csv module and pandas library. It correctly generated code to read the CSV file and compute mean, minimum, and maximum values for numerical columns. The outputs were accurate and easy to interpret. Gemini's explanation clarified how data is loaded and how statistical functions work. This task showed that Gemini is reliable for basic data analysis and is especially useful in Colab environments due to direct compatibility with pandas.

TASK 4:

 generate code for Bubble sort and Python's built-in sort() then at last Compare both implementations

#CODE:

```
[7]
✓ Os
# Step 1: Generate a sample list for sorting
import random

# Create a list of 1000 random integers for a meaningful comparison
sample_list = [random.randint(0, 10000) for _ in range(1000)]
print(f"Original list (first 10 elements): {sample_list[:10]}...")

▼
Original list (first 10 elements): [8347, 7887, 8169, 3918, 2994, 4731, 7994, 2741, 2844, 2418]...

[8]
✓ Os
# Step 2: Implement Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Make a copy of the sample list to sort with Bubble Sort
bubble_sorted_list = sample_list[:]
bubble_sorted_list = bubble_sort(bubble_sorted_list)
print(f"\nBubble Sorted list (first 10 elements): {bubble_sorted_list[:10]}...")

▼
...
Bubble Sorted list (first 10 elements): [4, 10, 11, 12, 12, 25, 36, 36, 50, 64]...
```

```
# Step 3: Use Python's built-in sort()

# Make another copy of the sample list to sort with built-in sort
python_sorted_list = sample_list[:]
python_sorted_list.sort() # In-place sort

# Alternatively, using sorted() creates a new sorted list:
# python_sorted_list_new = sorted(sample_list)

print(f"\nPython's built-in sorted list (first 10 elements): {python_sorted_list[:10]}...")
```

Python's built-in sorted list (first 10 elements): [4, 10, 11, 12, 12, 25, 36, 36, 50, 64]...

```
# Step 4: Compare both implementations (performance)
import timeit

# Setup for timeit
sorted_list_copy_bubble = sample_list[:]
sorted_list_copy_python = sample_list[:]

time_bubble = timeit.timeit("bubble_sort(arr)", globals={'bubble_sort': bubble_sort, 'arr': sorted_list_copy_bubble}, number=10)
time_python = timeit.timeit("arr.sort()", globals={'arr': sorted_list_copy_python}, number=10)

print(f"\nPerformance Comparison (average over 10 runs):")
print(f"Bubble Sort time: {time_bubble:.6f} seconds")
print(f"Python's built-in sort() time: {time_python:.6f} seconds")

# Verify if both lists are identical after sorting (they should be!)
if bubble_sorted_list == python_sorted_list:
    print("\nVerification: Both sorting methods produced identical results.")
else:
    print("\nVerification: The sorted lists are NOT identical. There might be an issue.")
```

```
...
Performance Comparison (average over 10 runs):
Bubble Sort time: 0.307319 seconds
Python's built-in sort() time: 0.000162 seconds

Verification: Both sorting methods produced identical results.
```

Code Explanation

1. Generate a Sample List (First Code Cell):

- `import random`: Imports the `random` module to generate random numbers.
- `sample_list = [random.randint(0, 10000) for _ in range(1000)]`: Creates a list of 1000 random integers between 0 and 10000. A larger list makes the performance difference more apparent.

2. Bubble Sort Implementation (Second Code Cell):

- `def bubble_sort(arr):`: Defines the Bubble Sort function.
- The outer loop (`for i in range(n)`) controls the number of passes. In each pass, the largest unsorted element "bubbles up" to its correct position.
- The inner loop (`for j in range(0, n - i - 1)`) iterates through the unsorted portion of the array.
- `if arr[j] > arr[j + 1]: arr[j], arr[j + 1] = arr[j + 1], arr[j]`: This is the core swap logic. If an element is greater than its neighbor, they are swapped.
- A copy of `sample_list` is made (`bubble_sorted_list = sample_list[:]`) to ensure the original list remains unchanged for the next sorting method.

3. Python's Built-in Sort (Third Code Cell):

- `python_sorted_list = sample_list[:]`: Another copy of the original list is made.
- `python_sorted_list.sort()`: This is the built-in list method that sorts the list *in-place*. Python's Timsort algorithm is highly optimized and efficient.
- (Commented out: `sorted(sample_list)` provides an alternative that returns a new sorted list without modifying the original.)

4. Performance Comparison (Fourth Code Cell):

- `import timeit`: Imports the `timeit` module, which is designed for measuring the execution time of small bits of Python code.
- `timeit.timeit(...)`: Measures the execution time of the provided code snippet.
 - `globals={...}`: Passes the necessary functions and variables to the execution environment of `timeit`.
 - `number=10`: Runs the code 10 times and returns the total time taken, which we then divide by 10 for an average.
- The output clearly shows the significant performance difference between the inefficient Bubble Sort and Python's highly optimized built-in sort.
- A verification step (`if bubble_sorted_list == python_sorted_list:`) is included to confirm that both methods indeed produce the same sorted result.

#OBSERVATION:

Gemini generated both bubble sort and Python's built-in `sort()` implementation efficiently. The bubble sort code demonstrated nested looping and step-by-step comparisons, while the built-in version provided a single-line solution leveraging Python's optimized algorithm (Timsort). The execution comparison clearly showed that built-in sorting is significantly faster and more readable. Gemini also explained time complexity differences when prompted. This task highlighted how AI tools can help understand algorithmic efficiency and simplify implementation for students.